T3 - Files, CSV and Exceptions (plus the "Student Height Logger")

Covers: text files (r/w/a), `with`, the `csv` module, create-if-missing-else-append, summary files, try/except/else/finally, and a COMPLETE solution to "Practice Exam Task 3". Earns marks in: 2024 Task 1a (load data), Practice Task 3 (replaces PyTorch, all 4 sub-tasks), and file/validation parts of 2021-2023.

---

1. What the exam asks

"Practice Task 3 (replacing pytorch): Working with CSV Files and Visualising Data in Python" - *Student Height Logger* (this is the shape the real Task 3 may take):

Part	Exact requirement (shortened)
Task 1 - CSV setup	Create `heights.csv` with columns `Name, Height`. **If it already exists do NOT overwrite - append.** Print a message when the file is created. Print a message when it is "open for appending".
Task 2 - Data entry	Prompt for name + height in cm (integer). `q` as the name stops. Each valid record is **appended** as a row. Prints `=== Task 2: Data Entry ===`, `Enter name (or 'q' to quit):`, `Enter height (cm):`, and finally `Data saved to heights.csv`.
Task 3 - Summary	Read the CSV into a DataFrame (pandas + NumPy). Print **total records, average, min, max**. Save them to `height_summary.txt`.
Task 4 - Plot	Histogram. Title `Height Distribution`, x-label `Height (cm)`, y-label `Frequency`. Re-running and typing `q` straight away must print **the same output every time**.

Test data the task insists you type: Amy 165, Ben 172, Ali 166, Yin 180, Mohammed 170, Gus 182, then `q`.

Other places this topic is examined - 2024 paper, Task 1a: "*Load the CSV file into a Pandas DataFrame*" *[2]* (reading a file; missing file = `FileNotFoundError`). - 2023 paper, Task 1b: validate input (year, empty fields) - needs `try/except ValueError`. - Lab 3 exercises 1-5 (menu + text file, quiz saved to CSV, Salaries.csv add/view/delete, sum numbers ignoring bad input). - tAPP-3 Problem 5-1: "*display file contents; if it does not exist print* `File not found. Please try again.` *and ask again*".

---

2. Mental model (analogy)

Idea	Analogy
A text file	A **paper notebook** in a drawer. Your program is the person who opens it.
`open(name, "r")`	**Read** - you may look but not write. The notebook must exist.
`open(name, "w")`	**Write** - you rip out all the pages and start a blank notebook. *Old data is gone.*
`open(name, "a")`	**Append** - you open at the last page and write on the next free line. Creates the notebook if missing.
`with open(...) as f:`	**Library loan**: borrow the notebook, and it is auto-returned when you leave the indented block - even after an error.
`os.path.exists(name)`	**Peeking into the drawer** to see whether the notebook is there before you decide to buy a new one.
CSV file	A notebook **ruled into columns**; commas are the column lines.
`csv.writer` / `csv.reader`	A **scribe** who writes a list as one ruled row / reads a row back as a list.
`DictReader` / `DictWriter`	Same scribe, but the **column headings are labels** - you say `row["Name"]`, not `row[0]`.
`try / except`	A **safety net under a trapeze**: try the trick; if you fall (error), the net catches you instead of crashing the show.

Idea	Analogy
`else` / `finally`	`else` = "only if you did NOT fall"; `finally` = "the cleanup crew, always comes".

3. Copy-paste templates

All of these were run and tested.

3.1 WHEN: you must read a text file (three styles - pick one)

```
with open("subjects.txt", "r") as f:       # "r" is the default mode
    text = f.read()                        # ONE string, includes \n
with open("subjects.txt") as f:
    lines = f.readlines()                  # LIST of lines, each ends \n
with open("subjects.txt") as f:
    for line in f:                         # best: one line at a time
        print(line.strip())                # strip() removes the \n
```

The lab-slide idiom also works: `line = f.readline()`, then `while line != "":` (an empty string means end of file) and call `readline()` again at the bottom of the loop.

3.2 WHEN: you must write or add to a text file

```
with open("subjects.txt", "w") as f:       # "w" WIPES the file first
    f.write("Maths\n")                     # write() wants a str + you add \n
with open("subjects.txt", "a") as f:       # "a" adds at the end
    f.write(f"Score: {95}\n")              # numbers -> f-string or str()
```

3.3 WHEN: "create the file if missing, otherwise append" (Task 1 pattern)

```
import csv
import os.path

FILE = "heights.csv"
if not os.path.exists(FILE):               # not there yet -> first run
    with open(FILE, "w", newline="") as f:
        csv.writer(f).writerow(["Name", "Height"])   # header row ONCE
    print("heights.csv was created")
else:                                      # already there -> keep data
    print("heights.csv is open for appending")
with open(FILE, "a", newline="") as f:     # always open in append mode
    csv.writer(f).writerow(["Amy", 165])
```

3.4 WHEN: you must read a CSV with the csv module

```
import csv
with open("heights.csv", "r") as f:
    for row in csv.reader(f):              # row is a LIST of strings
        print(row)                         # ['Amy', '165']
with open("heights.csv", "r") as f:
    for row in csv.DictReader(f):          # row is a DICT keyed by header
        print(row["Name"], int(row["Height"]))   # values are still str!
```

3.5 WHEN: you must write a CSV (list rows or dict rows)

```
import csv
with open("people.csv", "w", newline="") as f:     # newline="" always!
    writer = csv.DictWriter(f, fieldnames=["Name", "Age"])
    writer.writeheader()                           # Name,Age
    writer.writerow({"Name": "Ben", "Age": 20})
    writer.writerows([{"Name": "Ali", "Age": 21},
                      {"Name": "Yin", "Age": 22}])
```

3.6 WHEN: delete or edit one record (a CSV cannot be edited in place)

```
import csv
temp = []
with open("people.csv", "r") as f:         # 1. copy file -> list
```

```
    for row in csv.reader(f):
        temp.append(row)
temp = [row for row in temp if row[0] != "Ali"]    # 2. change the list
with open("people.csv", "w", newline="") as f:     # 3. overwrite file
    csv.writer(f).writerows(temp)
```

3.7 WHEN: "save the results to a text file"

```
scores = [90, 75, 60]
with open("summary.txt", "w") as f:
    f.write(f"Count: {len(scores)}\n")
    f.write(f"Average: {sum(scores) / len(scores):.1f}\n")
```

3.8 WHEN: user input might be bad / file might be missing

```
def read_number(prompt):
    """Keep asking until the user types a valid number."""
    while True:
        try:
            return float(input(prompt))          # may raise ValueError
        except ValueError:
            print("That is not a number, try again.")

fname = input("File name: ")
while True:
    try:
        inf = open(fname, "r")                   # may raise FileNotFound
    except FileNotFoundError:
        print("File not found. Please try again.")
        fname = input("File name: ")
    else:                                        # ran with NO error
        break
```

Full shape: `try:` risky code / `except ValueError:` / `except KeyError:` / `else:` only if no error / `finally:` always runs. Dictionary example: `stock["pear"]` raises `KeyError: 'pear'` (safer: `stock.get("pear", 0)`).

3.9 WHEN: the CSV is saved and you need statistics + a chart

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("heights.csv")                # file -> DataFrame
print(len(df), np.mean(df["Height"]), df["Height"].min())
plt.hist(df["Height"], bins=5, edgecolor="black")
plt.title("Height Distribution")
plt.xlabel("Height (cm)")
plt.ylabel("Frequency")
plt.show()
```

4. Line-by-line walkthrough - Practice Task 3 (complete solution)

Runnable file: `Practice/Solutions_py/practice_task3_height_logger.py` (the same code as below, joined together). Run it from the folder where `heights.csv` should live.

Plan: four functions (one per task) + `main()`. Functions make each task easy to mark and easy to re-run.

4.0 Imports and constants

```
import csv                      # standard library: read/write csv rows
import os.path                  # os.path.exists() = "is this file there?"
import pandas as pd             # DataFrame = spreadsheet in memory
import numpy as np              # fast maths: mean / min / max
import matplotlib.pyplot as plt  # drawing the histogram

CSV_FILE = "heights.csv"        # one place to change the file names
SUMMARY_FILE = "height_summary.txt"
```

Analogy: the imports are the tools you carry into the workshop. UPPER_CASE names are labels on the tool box: "never change while the program runs".

4.1 Task 1 - set up the file

```
def setup_csv(filename):
    """Create the csv with a header row if missing, else say we append."""
    if not os.path.exists(filename):       # first run (or file deleted)
        with open(filename, "w", newline="") as f:   # "w" = brand new
            writer = csv.writer(f)         # the scribe
            writer.writerow(["Name", "Height"])      # header row
        print(f"{filename} was created with columns: Name, Height")
    else:                                  # file already there: keep it
        print(f"{filename} already exists and is open for appending")
```

- `os.path.exists` peeks in the drawer. **False** -> buy a new notebook (`"w"` is safe here because there is nothing to destroy) and write the header once.
- **True** -> do *not* touch the file; just print the "open for appending" message. The real appending happens in Task 2.
- `newline=""` stops the csv module and Python both adding line breaks (blank rows on Windows).

4.2 Task 2 - data entry

```
def enter_data(filename):
    """Ask for name/height pairs until the user types 'q' as the name."""
    print("\n=== Task 2: Data Entry ===")
    with open(filename, "a", newline="") as f:   # "a" = add to the end
        writer = csv.writer(f)
        name = input("Enter name (or 'q' to quit): ").strip()
        while name.lower() != "q":               # 'q' or 'Q' stops
            try:
                height = int(input("Enter height (cm): "))   # whole cm
                if height <= 0:                  # sanity check
                    print("Height must be a positive whole number.")
                elif name == "":                 # blank name = invalid
                    print("Name cannot be empty.")
                else:
                    writer.writerow([name, height])   # valid -> saved
            except ValueError:                   # e.g. user typed 'abc'
                print("Invalid height - please enter a whole number.")
            name = input("Enter name (or 'q' to quit): ").strip()
    print(f"\nData saved to {filename}")
```

- **Loop shape (sentinel loop):** ask once *before* the loop, test, do work, ask again at the *bottom*. Like a bouncer who checks each guest at the door and shuts up shop when someone says the password `q`.
- `.strip()` removes accidental spaces; `.lower()` makes `Q` work too.

- `int(input(...))` converts text to a whole number. If the text is not a number Python raises `ValueError` - the `except` catches it so the program carries on (safety net).
- Only the **valid** branch calls `writerow`, so bad data never reaches the file.
- `print(...)` for "Data saved" is *outside* the `with` block: the file is closed (auto-returned) first - then we announce it.

4.3 Task 3 - summary statistics

```
def summarise(filename, summary_filename):
    """Read the csv with pandas, print + save the summary, return df."""
    print("\n=== Task 3: Summary Statistics ===")
    df = pd.read_csv(filename)                  # whole file -> table
    if df.empty:                                # header only, no rows
        print("No records yet, nothing to summarise.")
        return df
    heights = df["Height"].to_numpy()           # column -> numpy array
    total = len(df)                             # number of rows
    average = np.mean(heights)
    smallest = np.min(heights)
    largest = np.max(heights)

    lines = [                                   # build the text once ...
        f"Total records: {total}",
        f"Average height: {average:.1f} cm",
        f"Minimum height: {smallest} cm",
        f"Maximum height: {largest} cm",
    ]
    for line in lines:                          # ... print it ...
        print(line)
    with open(summary_filename, "w") as f:      # ... and save it
        f.write("\n".join(lines) + "\n")
    print(f"Summary saved to {summary_filename}")
    return df
```

- `pd.read_csv` re-reads **the whole file** (old rows + new rows). That is why quitting immediately gives identical output every run: nothing new was appended, the file is the same.
- `df.empty` guard: a file with only the header gives `mean = nan`; better to say so politely.
- Building `lines` once and then printing *and* writing the same list guarantees the screen and the file always match (one recipe, two plates).
- `"\n".join(lines)` glues the strings with a line break between them; we add a final `\n` so the file ends tidily.
- `"w"` is right here: the summary is regenerated from scratch every run.
- `return df` hands the table to the plotting function (no second `read_csv`).

4.4 Task 4 - histogram

```
def plot_histogram(df):
    """Histogram of the heights with the required title and labels."""
    if df.empty:                                # nothing to draw
        return
    plt.hist(df["Height"], bins=5, edgecolor="black")
    plt.title("Height Distribution")
    plt.xlabel("Height (cm)")
    plt.ylabel("Frequency")
    plt.show()
```

- A histogram sorts the heights into **bins** (buckets, like pigeonholes) and draws a bar as tall as the number of students in each bucket. `bins=5` with 6 values gives a readable picture; `edgecolor` separates the bars.
- Title and labels use the **exact** words from the task (copy them - the marker checks them).

4.5 Main program

```
def main():
    print("=== Task 1: CSV File Setup ===")
    setup_csv(CSV_FILE)
    enter_data(CSV_FILE)
```

```
    df = summarise(CSV_FILE, SUMMARY_FILE)
    print("\n=== Task 4: Visualisation ===")
    plot_histogram(df)


main()
```

4.6 What a run prints (first run, then re-run + `q`)

```
=== Task 1: CSV File Setup ===
heights.csv was created with columns: Name, Height

=== Task 2: Data Entry ===
Enter name (or 'q' to quit): Amy
Enter height (cm): 165
   ... (Ben 172, Ali 166, Yin 180, Mohammed 170, Gus 182) ...
Enter name (or 'q' to quit): q

Data saved to heights.csv

=== Task 3: Summary Statistics ===
Total records: 6
Average height: 172.5 cm
Minimum height: 165 cm
Maximum height: 182 cm
Summary saved to height_summary.txt
```

Second run (type only `q`): first line becomes `heights.csv already exists and is open for appending`, and the summary block is **identical** (6 records, 172.5). Delete `heights.csv` and re-run: "was created" appears again, and with no records you get "No records yet".

TIP: the task text does not dictate the Task 1 wording, so keep to *created* / *open for appending* - those are the two ideas the marker looks for.

5. Traps & debugging

Mistake	What you see	Fix
Opening an existing data file with `"w"`	No error - **your data silently vanishes**	Use `"a"` to add, `"r"` to read. Use `"w"` only to create/overwrite.
`open("x.csv")` before it exists	`FileNotFoundError: [Errno 2] No such file or directory: 'x.csv'`	Check `os.path.exists`, or catch the error, or create it with `"a"`/`"w"`.
`f.write(165)`	`TypeError: write() argument must be str, not int`	`f.write(str(165))` or an f-string.
Forgetting `\n` in `write`	All text jammed on one line	`f.write("text\n")` (print adds it for you, write does not).
Reading from a file opened with `"w"`/`"a"`	`io.UnsupportedOperation: not readable`	Re-open with `"r"`.
Using `f` after the `with` block	`ValueError: I/O operation on closed file.`	Keep all file work **inside** the indented block.
CSV numbers are strings	`sum(["165","172"])` gives `TypeError: unsupported operand type(s) for +: 'int' and 'str'`	`int(row[1])` / `float(...)`. (pandas converts for you.)
Missing `newline=""`	Blank line between rows (on Windows)	`open(..., newline="")` for every csv read/write.
Header written every run	`Name,Height` repeated in the middle of the file	Only write the header inside `if not os.path.exists(...)`.
`pd.read_csv` on an empty file (0 bytes)	`EmptyDataError: No columns to parse from file`	Write the header first (Task 1), or catch the error.
Mean of a header-only file	`nan`	Test `if df.empty:` first.
`int("4.5")` or `int("")`	`ValueError: invalid literal for int() with base 10: '4.5'`	Wrap in `try/except ValueError` and re-ask.
Bare `except:`	Hides **every** bug, even typos	Name the error: `except ValueError:`.
Print inside `try` after the risky line	Never printed when an error jumps out	Put "success" code in `else:`.
`input()` loop ends with `q` but `Q` typed	Program tries to convert `Q` as a name	`name.lower() != "q"`.
Deleting the first match only (`temp.remove`)	Duplicates remain	Use a list comprehension: `[r for r in temp if r[0] != name]`.
`plt.show()` before setting title/labels	Labels missing	Set title/labels **before** `plt.show()`.

NOTE (from the course solutions): `Lab3_sol/ex_1.py` crashes with `NameError: name 'filename' is not defined` if the user picks option 2 first (the file name is only set in option 1), and `ex_3.py`/`ex_4.py` crash on non-numeric menu input (`int(input())` has no `try`). Wrap menu input in `try/except ValueError` in the exam.

6. Anticipated exam questions

Q1 [3 marks] *Write code that creates* `heights.csv` *with columns Name and Height only if it does not already exist.*

```
import csv, os.path
if not os.path.exists("heights.csv"):
    with open("heights.csv", "w", newline="") as f:
        csv.writer(f).writerow(["Name", "Height"])
```

Q2 [4 marks] *Ask the user for name/height pairs until they type* `q`*, appending each valid record to the file. Reject heights that are not whole numbers.* Use template 4.2 (sentinel loop + `try/except ValueError` + `writerow` in `"a"` mode).

Q3 [3 marks] *Read the CSV into a DataFrame and print the number of records, the average, the minimum and the maximum height.*

```
df = pd.read_csv("heights.csv")
print("Total:", len(df), "Average:", np.mean(df["Height"]),
      "Min:", df["Height"].min(), "Max:", df["Height"].max())
```

Q4 [2 marks] *Save those statistics to* `height_summary.txt`*.*

```
with open("height_summary.txt", "w") as f:
    f.write(f"Total records: {len(df)}\n")
    f.write(f"Average height: {df['Height'].mean():.1f} cm\n")
```

Q5 [3 marks] *Plot a histogram titled "Height Distribution" with axis labels.* See template 3.9.

Q6 [2 marks] *Why does re-running and typing* `q` *at once give the same results?* Because the file is opened in append mode (never overwritten) and the summary is recomputed from the whole file; no new rows were added, so nothing changes.

Q7 [3 marks] *Prompt for a file name and print its contents. If it does not exist print "File not found. Please try again." and ask again.* Template 3.8 (`FileNotFoundError` loop; print `inf.read()` then `inf.close()` after the loop).

Q8 [3 marks] *Explain the difference between modes* `"r"`*,* `"w"` *and* `"a"`*.* r = read, file must exist; w = write, creates or **erases** existing content; a = append, creates if missing, writes after existing content.

Q9 [4 marks] *Delete the record for a given name from* `Salaries.csv`*.* Template 3.6 (copy to list, filter, rewrite with `"w"`). Print "Record not found." if the list length did not change.

Q10 [3 marks] *What do* `else` *and* `finally` *do in a try statement?* `else` runs only when the `try` block raised nothing; `finally` runs always (cleanup), error or not.

---

7. Quick-reference box (copy onto your notes sheet)

```
import csv, os.path        import pandas as pd, numpy as np
OPEN: open(f,"r") read | "w" WIPE+write | "a" add at end   (use `with`)
f.read() one str | f.readlines() list | for line in f: | line.strip()
f.write("text\n")  <- str only, add \n yourself
NEW-or-APPEND: if not os.path.exists(F): open(F,"w",newline="")+header
               then ALWAYS open(F,"a",newline="") to add rows
csv.writer(f).writerow([a,b]) | .writerows(list) | csv.reader(f) -> lists
csv.DictWriter(f,fieldnames=[..]) .writeheader() .writerow({..})
csv.DictReader(f) -> row["Name"]  (values are strings!)
CSV edit/delete: read -> temp list -> change list -> open "w" -> writerows
try: ... except ValueError: ... except FileNotFoundError: ...
     else: (no error) ... finally: (always)
input loop: x = input(); while x.lower() != "q": ...; x = input()
pandas: df = pd.read_csv(F) | len(df) | df["H"].mean()/.min()/.max()
        np.mean(arr) | df.empty | df["H"].to_numpy()
plot: plt.hist(df["H"],bins=5,edgecolor="black"); title; xlabel; ylabel; show
```